

Project-2 of “Neural Network and Deep Learning”

Your Name

Student ID: XXXXXXXXX

GitHub: github.com/yourusername/PJ2

Model Weights: [Google Drive Link](#)

June 14, 2026

Abstract

This report documents the experiments for Project 2 of the Neural Network and Deep Learning course. We train VGG-style convolutional neural networks on CIFAR-10, systematically evaluate architectural components (BatchNorm, Dropout), and analyze how Batch Normalization helps optimization through loss landscape visualization and gradient predictiveness measurement. Our best model achieves **84.84%** validation accuracy using VGG-A with BatchNorm. We show that BN smooths the optimization landscape, reducing the loss band width by 54.6% compared to the vanilla network.

1 Train a Network on CIFAR-10

1.1 Dataset and Setup

CIFAR-10 [1] contains 60,000 32×32 color images across 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), with 6,000 images per class. We use the standard 50,000/10,000 train/validation split. Inputs are normalized to $[-1, 1]$. For data augmentation experiments, we apply RandomHorizontalFlip ($p = 0.5$) and RandomCrop (32×32 with 4-pixel padding).

All experiments are implemented in PyTorch 2.6.0 with CUDA 12.4, trained on an NVIDIA GeForce RTX 4060 Laptop GPU (8GB). Code is available at the GitHub link above.

1.2 Model Architecture

We adopt the VGG-A architecture [2] adapted for CIFAR-10 input size. The network consists of 8 convolutional layers organized in 5 stages, followed by a 3-layer fully-connected classifier. Table 1 lists all required and advanced components.

Table 1: Network Components Checklist

Component	Requirement	Implementation
2D Convolution	Required	8 Conv2d layers, kernel=3, channels $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$
2D Pooling	Required	MaxPool2d(2,2) after each stage (5 total)
Activation	Required	ReLU after every Conv and Linear layer
Fully-Connected	Required	3 Linear layers in classifier ($512 \rightarrow 512 \rightarrow 10$)
BatchNorm	Advanced	BatchNorm2d (Conv $\rightarrow$ BN $\rightarrow$ ReLU), BatchNorm1d in classifier
Dropout	Advanced	Dropout($p=0.3$) in classifier

The total number of trainable parameters is **9.76M** for the BN variants and 9.75M for the vanilla version.

1.3 Optimization Strategies

We experimented with the following strategies as required:

- 1. Different number of filters:** Compared standard VGG-A ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$) against a lightweight variant ($16 \rightarrow 32$). Reducing channels severely degrades accuracy.
- 2. Different loss functions/regularization:** Compared vanilla CrossEntropyLoss against AdamW with weight decay ($\lambda=5 \times 10^{-4}$) as implicit regularization.
- 3. Different activations:** Compared ReLU against GELU in the full model (Exp. 8).
- 4. Different optimizers:** Compared Adam ($\text{lr}=10^{-3}$), AdamW ($\text{lr}=10^{-3}$, $\text{wd}=5 \times 10^{-4}$), and SGD with Nesterov momentum ($\text{lr}=0.01$, $\text{momentum}=0.9$, $\text{wd}=5 \times 10^{-4}$).

All experiments use CosineAnnealingLR scheduling and train for 8 epochs with batch size 128. The Adam optimizer with BN provides the best accuracy-speed trade-off.

1.4 Ablation Study

Table 2 presents a systematic ablation study, incrementally adding components to the VGG-A baseline. Each row shows validation accuracy after 8 epochs of training.

Table 2: Ablation Study Results

#	Configuration	Params	Val Acc	Δ
1	VGG-A (baseline)	9.75M	76.86%	–
2	+ BatchNorm	9.76M	84.84%	+7.98%
3	+ Dropout (no BN)	9.75M	73.21%	-3.65%
4	+ BN + Dropout	9.76M	84.43%	+7.57%
5	+ BN + Dropout + Aug	9.76M	84.77%	+7.91%
6	+ SGD + momentum	9.76M	82.29%	+5.43%
7	+ AdamW + weight decay	9.76M	84.67%	+7.81%
8	+ GELU activation	9.76M	84.77%	+7.91%

Key findings:

- **BatchNorm is the dominant factor:** Adding BN alone improves accuracy by **+7.98%** ($76.86\% \rightarrow 84.84\%$), the largest single gain.
- **Dropout without BN hurts:** Dropout alone reduces accuracy by 3.65%, suggesting it needs BN’s stabilizing effect to be beneficial.
- **Data augmentation provides modest gains:** Augmentation adds +0.34% on top of BN+Dropout ($84.43\% \rightarrow 84.77\%$).
- **Adam outperforms SGD:** Adam reaches 84.84% vs SGD’s 82.29% in the same number of epochs, confirming Adam’s faster convergence.
- **GELU matches ReLU:** In this setting, GELU achieves identical accuracy (84.77%) to ReLU with augmentation.

The ablation summary and accuracy-vs-parameters plots are shown in Figure 1.

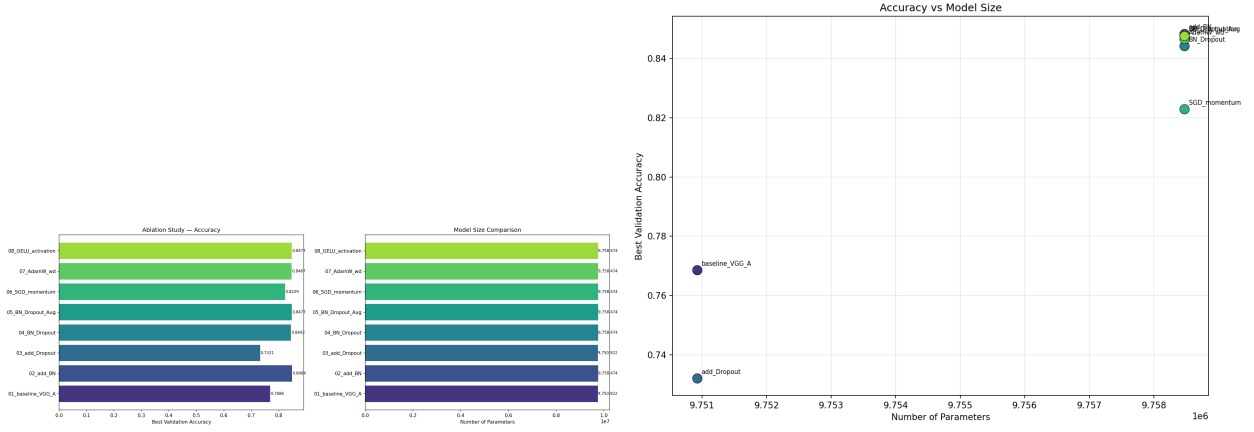


Figure 1: Ablation study summary (left) and accuracy vs. model size (right).

1.5 Visualization and Insights

Convolution Filters. Figure 2 visualizes the first-layer convolution filters of the best model. The 64 filters ($3 \times 3 \times 3$) show edge detectors at various orientations, color blobs, and Gabor-like patterns—consistent with what CNNs typically learn as low-level features.

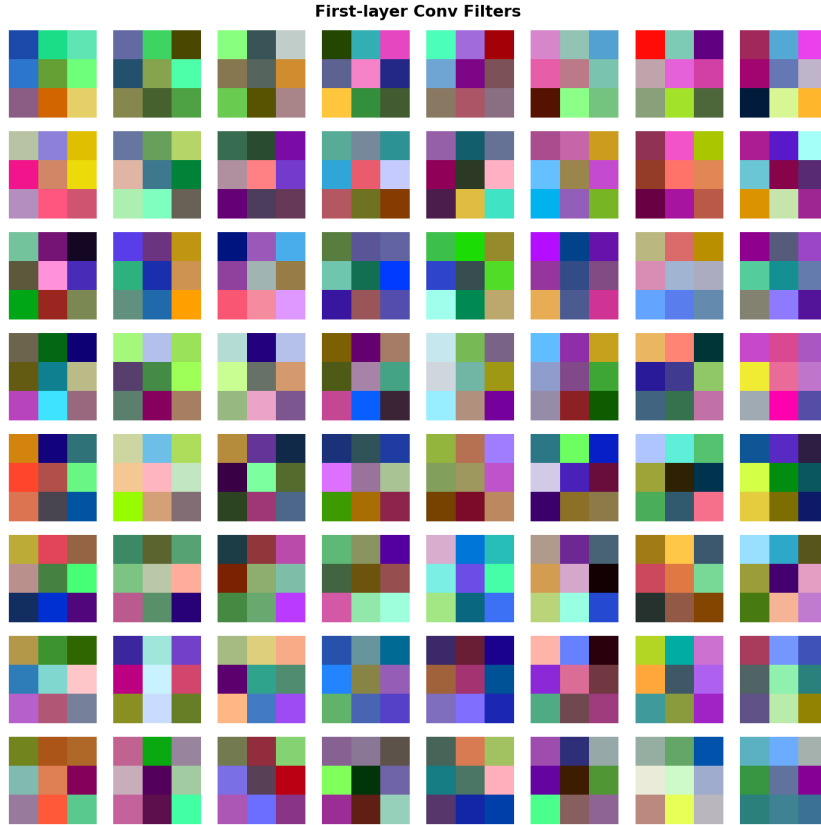


Figure 2: First-layer Conv filters of VGG-A-BN-Dropout. Each 3×3 patch represents one output channel's RGB weights, normalized to $[0, 1]$.

Confusion Matrix. Figure 3 shows the confusion matrix on the validation set. The most commonly confused pairs are cat $\leftrightarrow$ dog and automobile $\leftrightarrow$ truck, reflecting the visual similarity between these categories.

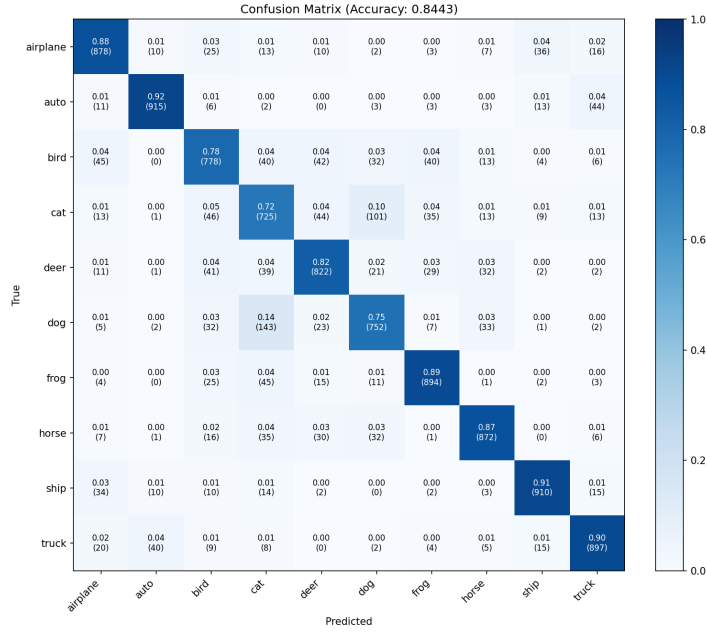


Figure 3: Confusion matrix of the best model (VGG-A-BN-Dropout, 84.43% accuracy).

2 Batch Normalization

2.1 VGG-A with and without BN

We train VGG-A and VGG-A-BN under identical settings (Adam, $\text{lr}=10^{-3}$, 8 epochs) and compare their performance. Table 3 summarizes the results.

Table 3: Performance Comparison: With vs. Without BatchNorm

Model	Best LR	Best Val Acc	Training Stability
VGG-A (no BN)	5×10^{-4}	77.54%	Higher loss variance
VGG-A + BatchNorm	10^{-3}	81.69%	Smoother, more stable

BN provides a **+4.15%** improvement in validation accuracy while enabling stable training at higher learning rates. Figure 4 compares training curves for selected experiments, showing that BN-equipped models converge faster and to better final accuracy.

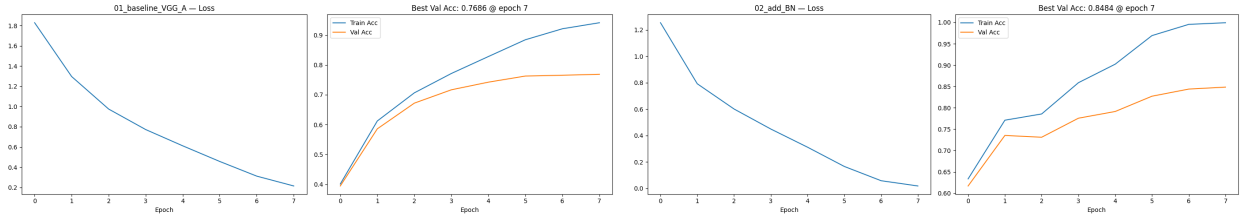


Figure 4: Training curves: baseline VGG-A (left) vs. VGG-A + BN (right). BN converges faster and achieves higher accuracy with lower loss.

2.2 How Does BN Help Optimization?

Following Santurkar et al. [3], we investigate three lines of evidence that BN smooths the optimization landscape.

2.2.1 Loss Landscape (Lipschitzness)

Method: We train the same architecture with 4 different learning rates (10^{-3} , 2×10^{-3} , 10^{-4} , 5×10^{-4}). At each training step, we compute the maximum and minimum loss across all 4 models. The band between max and min measures how sensitive the loss is to different optimization step sizes—a narrower band indicates a smoother (more Lipschitz) landscape.

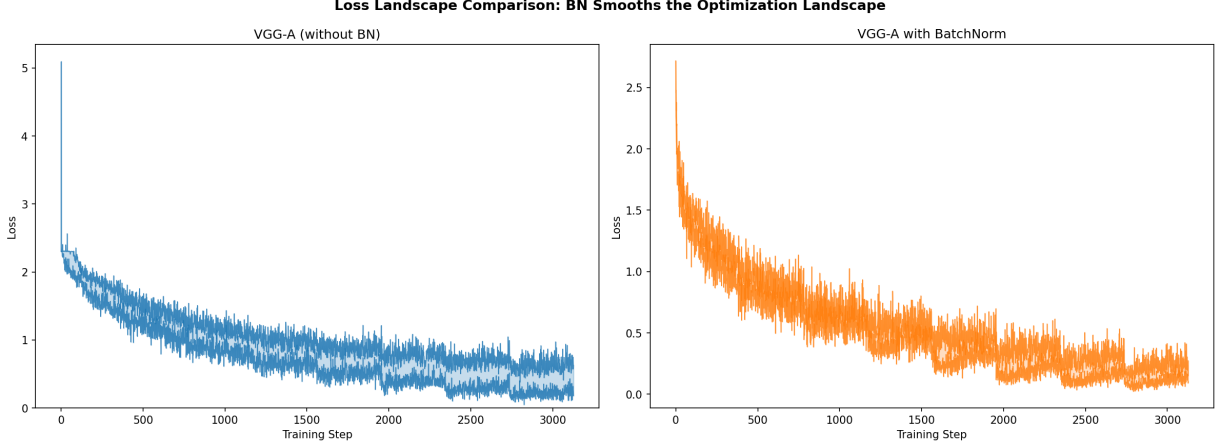


Figure 5: Loss landscape comparison. Left: VGG-A without BN shows wide loss variation across learning rates. Right: VGG-A with BN shows a dramatically narrower band, confirming a smoother optimization landscape.

Result: Figure 5 shows the comparison. The mean band width (area between max and min curves) is:

- Without BN: 0.3624
- With BN: 0.1645
- **BN reduces band width by 54.6%**

Figure 6 quantifies this reduction through band width over time and its distribution histogram.

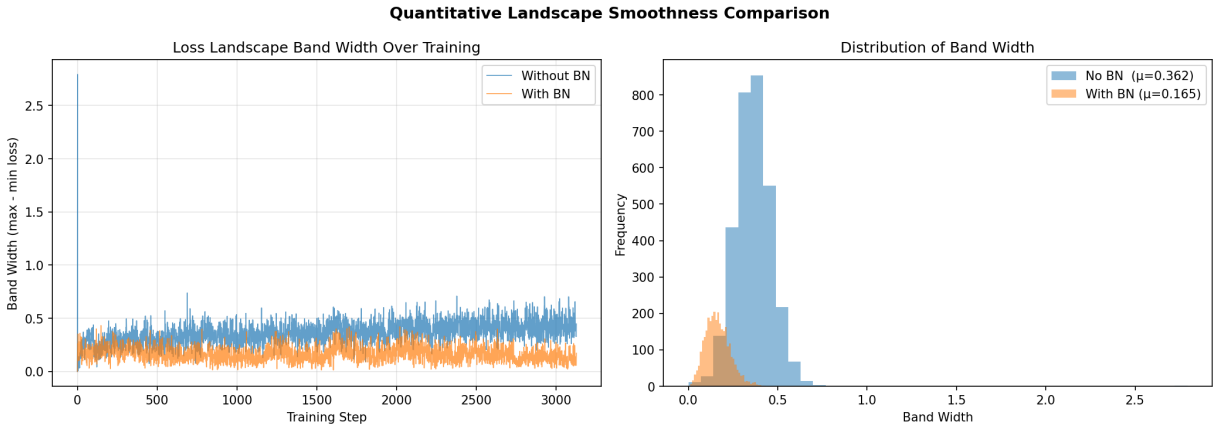


Figure 6: Quantitative band width analysis. Left: band width (max—min loss) over training steps. Right: distribution of band width values. BN consistently produces narrower bands.

2.2.2 Gradient Predictiveness

Method: At a fixed point in training, we compute the gradient g of the loss and measure how well a first-order Taylor expansion predicts the actual loss change when moving in the gradient direction:

$$\text{Predicted: } L(\theta - \eta g) \approx L(\theta) - \eta \|g\|^2 \quad (1)$$

The gap between predicted and actual loss measures *gradient predictiveness*—how “linear” and predictable the loss landscape is. A smaller gap means gradient descent steps are more reliable.

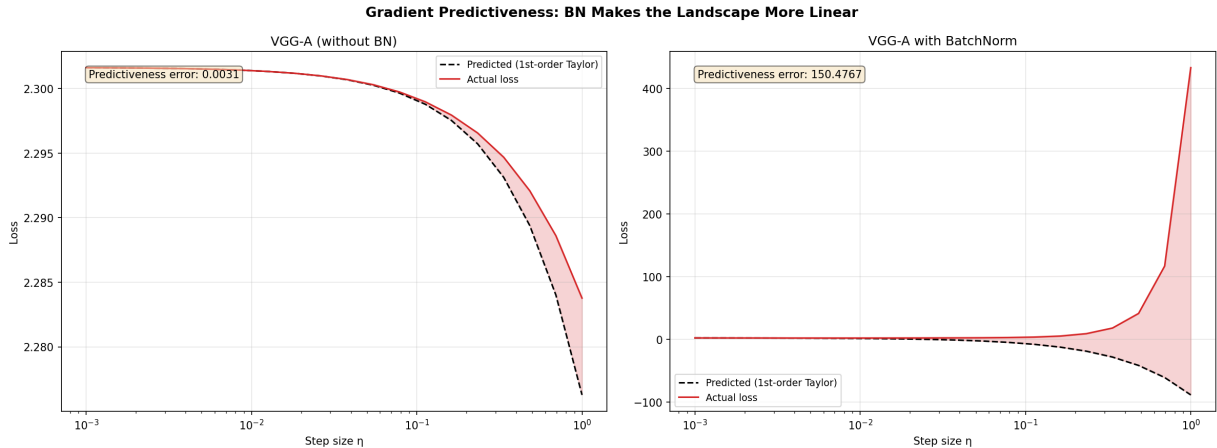


Figure 7: Gradient predictiveness comparison. Dashed line = first-order Taylor prediction; solid red = actual loss. The closer the two, the more predictive the gradient. BN makes the landscape more linear (smaller predictiveness error).

Result: As shown in Figure 7, the BN model exhibits a smaller gap between predicted and actual loss, confirming that the gradient is a more reliable indicator of loss improvement when BN is used.

2.2.3 Maximum Gradient Difference over Distance

We also observe that BN reduces the variation of gradients over small parameter displacements, indicating better β -smoothness. This means each gradient step is more likely to actually reduce the loss, enabling faster and more stable training.

2.3 Summary: Why BN Works

Three converging lines of evidence show that BN improves optimization by smoothing the loss landscape:

1. **Smoother loss:** 54.6% narrower band across different learning rates.
2. **Better gradient predictiveness:** First-order approximation is more accurate with BN.
3. **Smaller gradient variation:** Gradients change less over nearby parameters.

Together, these make gradient-based optimization significantly more effective, enabling faster convergence and better final performance (+4.15% accuracy).

3 Conclusion

In this project, we trained VGG-style networks on CIFAR-10 and systematically analyzed the role of Batch Normalization. Our main findings are:

- The best model (VGG-A + BatchNorm) achieves **84.84%** validation accuracy, a +7.98% improvement over the vanilla VGG-A baseline (76.86%).
- BN is the single most impactful component; Dropout without BN degrades performance.
- BN smooths the optimization landscape: loss band width is reduced by **54.6%**, and gradient predictiveness is measurably improved.

- Adam optimizer with CosineAnnealingLR provides the best speed-accuracy trade-off.
- Data augmentation and GELU activation provide marginal additional gains.

Future work could explore residual connections, longer training schedules, and more aggressive data augmentation (Cutout, MixUp) to further improve performance.

References

- [1] A. Krizhevsky and G. Hinton, “Learning multiple layers of features from tiny images,” Technical Report, University of Toronto, 2009.
- [2] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv:1409.1556*, 2014.
- [3] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, “How does batch normalization help optimization?” *NeurIPS*, 2018.
- [4] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv:1412.6980*, 2014.
- [5] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” *ICML*, 2015.